

|                        |                                        |
|------------------------|----------------------------------------|
| <b>Comenzado el</b>    | jueves, 16 de enero de 2025, 14:44     |
| <b>Estado</b>          | Finalizado                             |
| <b>Finalizado en</b>   | jueves, 16 de enero de 2025, 14:57     |
| <b>Tiempo empleado</b> | 13 minutos 1 segundos                  |
| <b>Puntos</b>          | 20,17/25,00                            |
| <b>Calificación</b>    | <b>8,07</b> de 10,00 ( <b>80,67%</b> ) |

**Pregunta 1**

Sin contestar

Se puntúa como 0 sobre 1,00

Sea el problema de la mochila entera con los objetos siguientes, donde la primera componente es el valor y la segunda el peso:  $(40, 2)$ ,  $(30, 5)$ ,  $(50, 10)$ ,  $(10, 5)$ . Si la capacidad de la mochila es 16, el tamaño máximo que alcanza la cola de prioridad en el algoritmo de ramificación y poda que utiliza el algoritmo voraz para calcular la prioridad de los nodos es:

Seleccione una:

- ☐ a. 6
- ☐ b. 4
- ☐ c. 5
- ☐ d. 2

Representando en cada nodo el valor, peso y estimación de la solución parcial por una tripla  $(valor, peso, estimación)$

1-Inicialmente en la cola solo está el nodo raíz  $(0, 0, 115)$ , así que tiene tamaño 1.

2- Sale ese nodo y entran  $nodo_1 = (40, 2, 115)$  y  $nodo_2 = (0, 0, 82)$ . El tamaño de la cola es 2.

3- Sale  $nodo_1$  y entran  $nodo_3 = (70, 7, 115)$  y  $nodo_4 = (40, 2, 98)$ . El tamaño de la cola es 3.

4- Sale  $nodo_3$  y entra  $nodo_5 = (70, 7, 80)$ . El tamaño de la cola es 3.

5- Sale  $nodo_4$  y entran  $nodo_6 = (90, 12, 98)$  y  $nodo_7 = (40, 2, 50)$ . El tamaño de la cola es 4.

6- Sale  $nodo_6$  y su hijo  $nodo_8 = (90, 12, 90)$  es la primera solución.

7- Puesto que el valor de esta solución es mayor o igual que las prioridades de los nodos que están en la cola, el algoritmo termina, quedando en la cola tres nodos:  $nodo_2$ ,  $nodo_5$  y  $nodo_7$ .

Por tanto la respuesta correcta es que el tamaño máximo que alcanza la cola de prioridad es 4.

La respuesta correcta es: 4

**Pregunta 2**

Correcta

Se puntúa 1,00 sobre 1,00

El *traspuesto* (o *inverso*) de un grafo  $G = (V, A)$  es otro grafo  $G^T = (V, A^T)$  donde  $A^T = \{(v, u) \mid (u, v) \in A\}$ .

¿Qué complejidad tendría un algoritmo para calcular el grafo traspuesto de un grafo dirigido representado mediante *listas de adyacentes* si el grafo tiene  $V$  vértices y  $A$  aristas?

Seleccione una:

- ☐ a.  $O(V^2)$
- ☐ b.  $O(V)$
- ☒ c.  $O(V + A)$  ✓
- ☐ d.  $O(V * A)$

Para calcular el grafo traspuesto podemos recorrer las listas de adyacentes de  $G$  y si encontramos el vértice  $v$  en la lista de adyacentes a  $u$ , añadir  $u$  a los adyacentes a  $v$  en el grafo  $G^T$ . Como hay  $V$  listas y el número total de elementos en todas ellas es  $A$ , el coste total está en  $O(V + A)$ .

La respuesta correcta es:  $O(V + A)$

**Pregunta 3**

Correcta

Se puntúa 1,00 sobre 1,00

Dado un grafo dirigido implementado mediante una *matriz de adyacencia*, ¿cuál sería el coste de un algoritmo eficiente que busque un vértice con grado de entrada 0?

Seleccione una:

- ☒ a.  $O(V * V)$  ✓ Cierto. Para encontrar un nodo con grado de entrada 0 tenemos que recorrer las columnas de la matriz hasta dar con una que sea entera de 0's. Por lo tanto el coste es cuadrático en el número de vértices.
- ☐ b.  $O(1)$
- ☐ c.  $O(V + A)$
- ☐ d. Ninguna de las anteriores.

- a. Cierto. Para encontrar un nodo con grado de entrada 0 tenemos que recorrer las columnas de la matriz hasta dar con una que sea entera de 0's. Por lo tanto el coste es cuadrático en el número de vértices.
- b. Falso. Incorrecta. Es cuadrática con respecto al número de vértices independientemente de cuántas aristas haya.
- c. Falso. Incorrecta. Es cuadrática con respecto al número de vértices independientemente de cuántas aristas haya.
- d. Falso. La respuesta correcta es:  $O(V * V)$

La respuesta correcta es:  $O(V * V)$

**Pregunta 4**

Correcta

Se puntúa 1,00 sobre 1,00

El problema de la multiplicación encadenada de  $n$  matrices de forma óptima puede resolverse con una complejidad en tiempo en:

Seleccione una:

- ☐ a.  $O(n^2)$
- ☐ b.  $O(m^2)$  siendo  $m$  el número de filas y columnas de las  $n$  matrices
- ☐ c.  $O(m^3)$  siendo  $m$  el número de filas y columnas de las  $n$  matrices
- ☒ d.  $O(n^3)$  ✓

El algoritmo calcula para cada pareja de índices  $i, j$  tales que  $i \leq j$  el número mínimo de multiplicaciones básicas para multiplicar las matrices de la  $i$  a la  $j$ . El cálculo de cada valor es lineal en  $n$  porque requiere considerar las distintas formas de colocar los paréntesis externos para realizar la última multiplicación. Puesto que la cantidad de parejas a calcular es del  $O(n^2)$ , el coste está en  $O(n^3)$ .

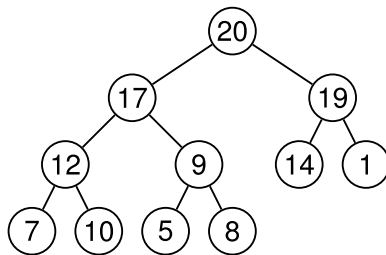
La respuesta correcta es:  $O(n^3)$

**Pregunta 5**

Correcta

Se puntúa 1,00 sobre 1,00

En este montículo de máximos, ¿qué valores pueden haber sido el último en insertarse?



Seleccione una:

- ☐ a. Solamente el 8.
- ☐ b. 20, 17, 9 y 8.
- ☐ c. Podría ser cualquiera.
- ☒ d. 9 y 8. ✓

El último elemento insertado se añadió en la hoja más a la derecha del último nivel y después fue flotado si hacía falta hacia la raíz. Puede haber sido el 8 y no haber necesitado ser flotado. Puede haber sido el 9 y haberse intercambiado con el 8, que ocupaba su posición. Pero no puede ser el 17, porque entonces el 9 ocuparía su posición antes de la última inserción, y el 9 no puede ser padre del 12 en un montículo de máximos. Por el mismo motivo no puede ser el 20 el último elemento insertado.

La respuesta correcta es: 9 y 8.

**Pregunta 6**

Correcta

Se puntúa 1,00 sobre 1,00

Indica el valor de la solución óptima que devuelve el algoritmo voraz que resuelve el problema de la mochila real en el caso en que el peso límite es 50 y los objetos tienen los siguientes pesos y valores:

| Objetos | Valores | Pesos |
|---------|---------|-------|
| 1       | 60      | 10    |
| 2       | 100     | 20    |
| 3       | 120     | 30    |

Seleccione una:

- ☐ a. 180
- ☒ b. 240 ✓
- ☐ c. 220
- ☐ d. 230

Aplicando la estrategia voraz, se consideran por orden los objetos 1, 2 y por último el 3. Los objetos 1 y 2 caben completos y del objeto 3 solamente cabe 2/3 por lo que el valor es  $60 + 100 + 2 * 120/3 = 240$ .

La respuesta correcta es: 240

**Pregunta 7**

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos que tenemos una estructura de partición con  $N$  elementos que utiliza *unión rápida por tamaño y compresión de caminos*. La estructura empieza vacía (partición trivial) y después se realizan  $N^2$  operaciones de unión aleatorias. A continuación, se realizan operaciones para saber si están unidos o no cada posible par de elementos (aproximadamente  $N^2/2$ ). El orden de complejidad de la altura de los árboles en la estructura resultante es:

Seleccione una:

- ☒ a.  $O(1)$  ✓
- ☐ b.  $O(N)$
- ☐ c.  $O(\log N)$
- ☐ d.  $O(N^2)$

Al haber compresión de caminos, al buscar un elemento este termina estando en la raíz de un árbol o en la raíz de un hijo de la raíz de un árbol, es decir, a profundidad 1 o 2. Si se realizan búsquedas con todos los elementos, todos los árboles terminan teniendo altura 1 o 2.

La respuesta correcta es:  $O(1)$

**Pregunta 8**

Correcta

Se puntúa 1,00 sobre 1,00

Indica cuáles de los siguientes algoritmos sobre grafos son voraces:

Seleccione una o más de una:

- ☐ a. Algoritmo de búsqueda en profundidad.
- ☒ b. Algoritmo de Dijkstra. ✓
- ☐ c. Algoritmo de búsqueda en anchura.
- ☒ d. Algoritmo de Kruskal. ✓

Los algoritmos de Kruskal y Dijkstra son voraces. Las decisiones que toman nunca se reconsideran y se garantizan que conducen a la solución óptima. El algoritmo de Kruskal encuentra el árbol de recubrimiento mínimo de un grafo valorado seleccionando las aristas por orden creciente de coste, siempre que no creen ciclos, hasta obtener un árbol de recubrimiento. El algoritmo de Dijkstra calcula los caminos mínimos en un digrafo valorado desde un vértice origen a todos los demás considerando los vértices en orden creciente de distancia al origen.

Sin embargo, los algoritmos de búsqueda en profundidad y anchura exploran de forma sistemática el grafo hasta explorarlo por completo, o bien hasta encontrar el objeto de la búsqueda. Si un camino de exploración no tiene éxito continúan la búsqueda por otro.

Las respuestas correctas son: Algoritmo de Dijkstra., Algoritmo de Kruskal.

**Pregunta 9**

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuántas formas distintas hay de multiplicar secuencialmente  $n$  matrices de dimensiones compatibles?

Seleccione una:

- ☐ a. Depende de las dimensiones de cada una de las matrices.
- ☐ b. Depende de los valores de cada una de las matrices.
- ☒ c.  $(n - 1)!$  ✓
- ☐ d.  $n^3$

Si llamamos  $m_n$  a las formas distintas de multiplicar  $n$  matrices,  $m_2 = 1$  porque solo hay una forma de multiplicar dos matrices, independientemente de sus dimensiones (son compatibles) y de sus valores. Si  $n \geq 3$ , tendremos que multiplicar un par contiguo de esas matrices de  $n - 1$  posibles a elegir y eso deja una secuencia de  $n - 1$  matrices que tendremos que seguir multiplicando, es decir,  $m_n = (n - 1) m_{n-1}$ . Luego  $m_n = (n - 1)!$ .

La respuesta correcta es:  $(n - 1)!$

**Pregunta 10**

Sin contestar

Se puntúa como 0 sobre 1,00

Los grafos con aristas de valores negativos:

Seleccione una:

- ☐ a. Tienen caminos de coste mínimo entre cada par de vértices si solo hay una arista con valor negativo.
- ☐ b. Tienen caminos de coste mínimo entre cada par de vértices si no contienen ciclos de coste negativo.
- ☐ c. No tienen caminos de coste mínimo entre cada par de vértices.
- ☐ d. Tienen caminos de coste mínimo entre cada par de vértices si a lo sumo hay dos aristas con valor negativo.

El algoritmo de Bellman-Ford resuelve el problema de encontrar los caminos mínimos desde un vértice origen a todos los demás si desde ese vértice no se puede alcanzar ningún ciclo de coste negativo, con coste en  $O(V * A)$ .

La respuesta correcta es: Tienen caminos de coste mínimo entre cada par de vértices si no contienen ciclos de coste negativo.

**Pregunta 11**

Correcta

Se puntúa 1,00 sobre 1,00

La versión que optimiza la cantidad de memoria utilizada del algoritmo de programación dinámica ascendente que calcula un número combinatorio acaba de rellenar el vector C que corresponde a los números combinatorios  $\binom{7}{0} \binom{7}{1} \dots \binom{7}{8}$  y queda de la siguiente forma:

|   |   |    |    |    |    |   |   |   |
|---|---|----|----|----|----|---|---|---|
| 0 | 1 | 2  | 3  | 4  | 5  | 6 | 7 | 8 |
| 1 | 7 | 21 | 35 | 35 | 21 | 7 | 1 | 0 |

Indica qué valores de este vector suma el algoritmo para calcular  $\binom{8}{5}$

Seleccione una:

- ☐ a. Con este vector aún no se puede calcular
- ☐ b.  $C[3] + C[4] = 35 + 35$
- ☒ c.  $C[4] + C[5] = 35 + 21$  ✓
- ☐ d.  $C[3] + C[5] = 35 + 21$

Para calcular  $\binom{8}{5}$  son necesarios los valores  $\binom{7}{4}$  y  $\binom{7}{5}$ , situados en las posiciones 4 y 5 respectivamente por lo que la respuesta es  $C[4] + C[5] = 35 + 21$ .

La respuesta correcta es:  $C[4] + C[5] = 35 + 21$

**Pregunta 12**

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es el coste de añadir una arista en un grafo no dirigido con  $V$  vértices y  $A$  aristas, representado mediante una *matriz de adyacencia*?

Seleccione una:

- ☐ a.  $O(V + A)$
- ☒ b.  $O(1)$  ✓
- ☐ c.  $O(V * A)$
- ☐ d.  $O(V)$

Para añadir la arista  $u - v$  basta con cambiar dos posiciones de la matriz ( $G[u][v]$  y  $G[v][u]$ ) a cierto.

La respuesta correcta es:  $O(1)$

**Pregunta 13**

Correcta

Se puntúa 1,00 sobre 1,00

En el algoritmo de Floyd para calcular los caminos mínimos entre cada par de vértices la matriz se puede rellenar en cada iteración:

Seleccione una:

- ☒ a. de cualquier forma. ✓
- ☐ b. solamente de arriba abajo y de derecha a izquierda.
- ☐ c. solamente por diagonales paralelas a la principal comenzando en la diagonal encima de la principal.
- ☐ d. solamente de arriba abajo y de izquierda a derecha.

En la iteración  $k$ -ésima cada posición  $(i, j)$  de la matriz necesita los valores  $(i, k)$ ,  $(k, j)$  y el propio  $(i, j)$  de la iteración anterior. Pero se puede demostrar que en la iteración  $k$ -ésima los valores de la fila  $k$  y de la columna  $k$  no se modifican, por lo que la matriz se puede rellenar de cualquier forma sin que se pierdan los valores que se necesitan en cada momento.

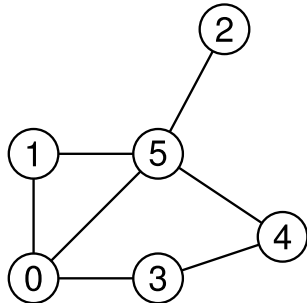
La respuesta correcta es: de cualquier forma.

**Pregunta 14**

Correcta

Se puntúa 1,00 sobre 1,00

¿En qué orden se visitarían los vértices de este grafo si realizamos un recorrido en profundidad desde el vértice 3? Escribe los identificadores de los vértices separados por espacios en el orden en que son visitados. Supón que los vértices en las listas de adyacentes están ordenados de menor a mayor.



Respuesta: 3 0 1 5 2 4



Los vértices se recorren en este orden: 3 0 1 5 2 4.

La respuesta correcta es: 3 0 1 5 2 4

**Pregunta 15**

Parcialmente correcta

Se puntúa 0,50 sobre 1,00

Hemos intentado resolver un problema,  $P$ , con un algoritmo que aplica estrategia voraz, pero al intentar demostrar que esa estrategia es correcta aplicando en método de reducción de diferencias, llegamos a probar que la solución voraz puede ser peor que una solución óptima.

¿Qué podemos afirmar al respecto?

Seleccione una o más de una:

- ☐ a. La estrategia voraz no se puede demostrar con el método de reducción de diferencias, habría que usar otra técnica de demostración.
- ☐ b. El problema  $P$  no se puede resolver con una estrategia voraz.
- ☒ c. Nuestra estrategia voraz es incorrecta. Cierto. Nuestra estrategia no produce una solución óptima.
- ☐ d. No sabemos si el problema  $P$  se puede resolver con una estrategia voraz o no.
- ☐ e. Hemos elegido mal la solución óptima.

- a. Falso. Si al tratar de aplicar el método de reducción de diferencias vemos que la solución voraz puede ser peor que otra solución válida, entonces es que la solución voraz no es óptima por definición y no se puede probar lo contrario.
- b. Falso. Puede ser que exista otra estrategia voraz que sí que funcione.
- c. Cierto. Nuestra estrategia no produce una solución óptima.
- d. Cierto. No podemos afirmar que se pueda (no hemos encontrado una estrategia voraz), pero tampoco sabemos si no se puede (puede existir una estrategia voraz correcta para resolverlo).
- e. Falso. La solución óptima no la hemos "elegido", solo hemos asumido que existe una solución óptima.

Las respuestas correctas son: Nuestra estrategia voraz es incorrecta., No sabemos si el problema  $P$  se puede resolver con una estrategia voraz o no.



**Pregunta 16**

Incorrecta

Se puntúa -0,33 sobre 1,00

Si el TAD de conjuntos disjuntos se implementa con la versión de *búsqueda rápida*, ¿cuál es el coste de la operación **unir** en el caso peor si el conjunto tiene  $N$  elementos?

Seleccione una:

- ☐ a.  $O(N^2)$
- ☒ b.  $O(\log N)$  ✖
- ☐ c.  $O(N)$
- ☐ d.  $O(1)$

Al unir dos conjuntos hay que buscar en todo el vector para mover los elementos de un conjunto al otro.

La respuesta correcta es:  $O(N)$

**Pregunta 17**

Correcta

Se puntúa 1,00 sobre 1,00

La función

```
void funcion(vector<int> & v) {
    for (int i = 1; i < v.size(); ++i) {
        hundir_max(v, v.size(), i);
    }
}
```

Seleccione una:

- ☒ a. mueve los elementos del vector, pero no lo convierte necesariamente en un montículo. ✔
- ☐ b. convierte un vector cualquiera en un montículo.
- ☐ c. convierte un montículo de máximos en uno de mínimos.
- ☐ d. siempre deja el vector sin modificar.

Un vector puede convertirse en un montículo de dos maneras distintas:

- recorriendo los elementos de izquierda a derecha y *flotando* cada elemento entre los anteriores (ya procesados), o
- recorriendo la primera mitad de los elementos de derecha a izquierda, *hundiendo* cada elemento entre los siguientes, que ya forman montículos.

Aquí se han mezclado las dos ideas, lo que no garantiza que el resultado final sea un montículo, aunque haya elementos que cambien de lugar.

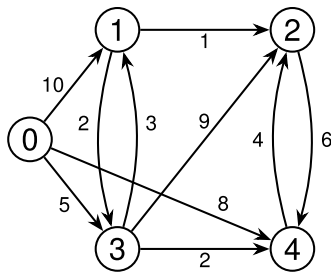
La respuesta correcta es: mueve los elementos del vector, pero no lo convierte necesariamente en un montículo.

**Pregunta 18**

Correcta

Se puntúa 1,00 sobre 1,00

Supón que ejecutamos el algoritmo de Dijkstra sobre este grafo utilizando como origen el vértice 0.



¿Cuál es el vértice anterior a 2 en su camino mínimo?

Seleccione una:

- ☐ a. 3
- ☒ b. 1 ✓
- ☐ c. 4
- ☐ d. Ninguno, porque no es alcanzable desde el origen.

El camino mínimo del vértice 0 al vértice 2 es  $0 \rightarrow 3 \rightarrow 1 \rightarrow 2$ . El penúltimo vértice es el 1.

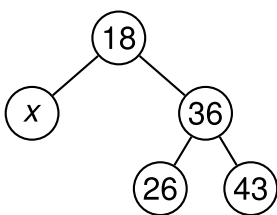
La respuesta correcta es: 1

**Pregunta 19**

Correcta

Se puntúa 1,00 sobre 1,00

Si al insertar consecutivamente los elementos 5 y 13 en el siguiente árbol AVL no se ha realizado ninguna rotación, ¿cuál es el menor valor posible de  $x$ ?



Respuesta: 6



Como los valores a insertar (5 y 13) son menores que 18, tendrán que colocarse necesariamente como descendientes del nodo  $x$ . En función del orden relativo de  $x$ , 5 y 13 se dan tres situaciones posibles:

1. Si  $5 < x < 13$ , cada valor acaba a un lado de  $x$  y no se produce ninguna rotación.
2. Si  $x$  es menor que 5, se necesita una rotación simple a izquierda para reequilibrar el nodo  $x$ .
3. Si  $x$  es mayor que 13, se necesita una rotación doble izquierda-derecha para reequilibrar el nodo  $x$ .

En conclusión, se tiene que dar la primera de las situaciones para evitar una rotación y entonces el menor valor posible de  $x$  es 6.

La respuesta correcta es: 6

**Pregunta 20**

Sin contestar

Se puntúa como 0 sobre 1,00

En el problema de la mochila real, si hay un subconjunto de objetos cuya suma de pesos es igual al peso límite, con seguridad dicha solución es óptima.

Seleccione una:

- ☐ a. Verdadero
- ☐ b. Falso

Falso. Supongamos peso límite 4 y dos objetos con valores {4, 10} y respectivos pesos {4, 2}. El subconjunto formado únicamente por el primer objeto alcanza exactamente el peso límite. Sin embargo no es una solución óptima, ya que su valor es 4 mientras que la solución formada por el segundo objeto completo y la mitad del primero alcanza el valor 12.

Las soluciones óptimas del problema de la mochila real siempre llenan la mochila pero no todas las soluciones que la llenan son óptimas.

La respuesta correcta es: Falso

**Pregunta 21**

Correcta

Se puntúa 1,00 sobre 1,00

Dado un vector ordenado de menor a mayor, ¿cuál es el coste de convertirlo en un montículo de máximos?

Seleccione una:

- ☐ a.  $O(1)$ .
- ☐ b. No está garantizado que se pueda convertir en un montículo de máximos.
- ☐ c.  $O(N \log N)$
- ☒ d.  $O(N)$ . ✓

Sí que se puede convertir en un montículo de máximos. Ahora bien, si el vector está ordenado de menor a mayor, cada elemento (en la primera mitad del vector) es menor que todos los siguientes por lo que tendrá que ser hundido hasta una hoja. Esto quiere decir que el coste será de  $O(N)$ .

La respuesta correcta es:  $O(N)$ .

**Pregunta 22**

Correcta

Se puntúa 1,00 sobre 1,00

En un problema de minimización resuelto mediante ramificación y poda la estimación utilizada para asignar prioridad a los nodos:

Seleccione una:

- ☐ a. debe ser una cota superior de la mejor solución alcanzable desde ese nodo.
- ☐ b. siempre coincide con el valor de la solución si es una solución completa.
- ☒ c. debe ser una cota inferior de la mejor solución alcanzable desde ese nodo. ✓
- ☐ d. siempre puede ser el valor de la solución parcial construida hasta el momento.

En un problema de minimización la estimación utilizada para asignar prioridad a los nodos ha de ser una cota inferior, no superior, de la mejor solución alcanzable desde ese nodo. De esta forma si la estimación de un nodo es superior al valor de la mejor encontrada hasta el momento eso quiere decir que ninguna solución alcanzada desde ese nodo puede mejorarla y no interesa por tanto expandir dicho nodo.

El valor de la solución parcial no tiene por qué ser siempre cota inferior de la mejor solución alcanzable, por lo que esta opción no es correcta. Si se trata de una solución completa, en general tampoco su valor tiene por qué coincidir con la estimación, aunque es bastante habitual.

La respuesta correcta es: debe ser una cota inferior de la mejor solución alcanzable desde ese nodo.

**Pregunta 23**

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es el coste, en el caso peor, de insertar un nuevo elemento en una cola de prioridad de máximos con  $N$  elementos implementada mediante un montículo de máximos?

Seleccione una:

- ☐ a.  $O(N \log N)$
- ☐ b.  $O(1)$
- ☒ c.  $O(\log N)$  ✓
- ☐ d.  $O(N)$

El nuevo elemento se coloca en el último nivel y es flotado hacia la raíz, en el caso peor la altura del árbol, que es logarítmica respecto al número de nodos.

La respuesta correcta es:  $O(\log N)$

**Pregunta 24**

Correcta

Se puntúa 1,00 sobre 1,00

El coste de recorrer completamente un AVL con  $N$  nodos para producir una lista ordenada de sus elementos está en

Seleccione una:

- ☐ a.  $O(1)$
- ☐ b.  $O(N \log N)$
- ☐ c.  $O(\log N)$
- ☒ d.  $O(N)$  ✓

Al ser un árbol binario de búsqueda, para producir una lista ordenada basta con recorrer el árbol en inorden, lo que tiene un coste lineal respecto al número de nodos del árbol.

La respuesta correcta es:  $O(N)$

**Pregunta 25**

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos que el algoritmo de Floyd va a empezar a rellenar la matriz  $k$ -ésima, es decir, la que corresponde a considerar los vértices del 1 al  $k$  como vértices intermedios. Indica qué afirmaciones son correctas en un grafo sin ciclos de coste negativo:

Seleccione una o más de una:

- ☒ a. La columna  $k$  no cambia. ✓
- ☒ b. La fila  $k$  no cambia. ✓
- ☒ c. La diagonal principal no cambia. ✓
- ☒ d. Para rellenar cada posición se necesitan a lo sumo tres posiciones de la matriz  $(k-1)$ -ésima. ✓

Se puede demostrar que en la iteración  $k$ -ésima los valores de la fila  $k$  y de la columna  $k$  no se modifican, ya que poder pasar por el vértice  $k$  para construir caminos desde o hasta  $k$  no permite reducir el coste de los caminos mínimos obtenidos con los vértices 1 a  $k-1$  como intermedios si no hay ciclos de coste negativo.

Los valores de las diagonales valen 0 en todas las iteraciones si no hay ciclos de coste negativo.

En la iteración  $k$ -ésima cada posición  $(i, j)$  de la matriz necesita los valores  $(i, k)$ ,  $(k, j)$  y el propio  $(i, j)$  de la matriz  $(k-1)$ -ésima.

Las respuestas correctas son: La columna  $k$  no cambia., La fila  $k$  no cambia., La diagonal principal no cambia., Para rellenar cada posición se necesitan a lo sumo tres posiciones de la matriz  $(k-1)$ -ésima.